Uruchomienie i połączenie stacku LGTM z aplikacją FastAPI.

Monitoring działania aplikacji jest niezbędnym narzędziem w rękach osób zajmujących się utrzymaniem stabilnego działania programów. Jego głównym zadaniem jest zbudowanie wiedzy o potencjalnych przebiegach zapytań i danych w programie, w taki sposób, aby przy wystąpieniu potencjalnego problemu, móc zidentyfikować szybciej jego przyczynę oraz miejsce wystąpienia w kodzie. Zdobywanie tej wiedzy opiera się o analizę sygnałów. Trzy główne sygnały (tzw. filary) wykorzystywane w monitoringu to: logi, metryki i trace'y.

Mimo całkiem sporej oferty narzędzi, zarówno komercyjnych jak i open source, oferta edukacyjna w tym zakresie jest mocno ograniczona. Większość z dostępnych materiałów mówi o tym, w jaki sposób użyć narzędzia X do odczytania i zaprezentowania wymiennie - logów, metryk lub trace'ów, lub ich wspólnej kombinacji. Właściwie niespotykane jest wytłumaczenie dobrych praktyk stosowania np. metryk w jakimś konkretnym przypadku - np. transferu danych. W wypadku sporej standaryzacji podejścia do tworzenia oprogramowania wydaje się to aż nieprawdopodobne, jednak dalej jest to pewnego rodzaju wiedza tajemna.

Przyczyn takiego stanu rzeczy można doszukiwać się w różnych implementacjach pewnych standardów stosowanych przez producentów tych narzędzi. Kolejnym powodem może być specyficzne ich zastosowanie przez klientów - każdy klient może je implementować w inny sposób i mieć różne potrzeby. Dodatkowo, monitoringiem zajmują się częściej osoby związane z infrastrukturą, niż produkcją kodu. Są mniej liczną grupą w porównaniu z programistami, co może mieć wpływ na ilość materiałów do nauki.

Problem standaryzacji został rozwiązany w roku 2019 po wygaszeniu dwóch projektów związanych z trace'ami - OpenTracing i OpenCensus. W wyniku tego powstał projekt OpenTelemetry, który stał się de facto standardem składni trace'ów oraz, trochę później, metryk i logów.

Pozostaje kwestia dostępu do praktycznej wiedzy o tym co można uzyskać z logów, metryk i trace'ów. W tym dokumencie zostanie nakreślona odpowiedź na pytanie:

Po co mi to wszystko?

Zanim jednak to nastąpi - potrzebujemy narzędzi do wizualizacji sygnałów. Nasz wybór padł na zestaw Looks Good To Me(LGTM) dostarczony przez firmę Grafana, ze względu na to, że jest to rozwiązanie kompleksowe - łączy wszystkie sygnały w 1 narzędziu.

1. Wymagania.

- Docker
- Python + PiP
- Wolne porty na localhost: *3000, 3100, 4317, 4318, 9090*

2. Omówienie stacku LGTM.

L – Loki – Obsługa logów. Operuje na porcie *3100*.

G – Grafana – Główny interfejs całego stacku LGTM. Wyświetla dane zebrane przez inne serwisy. Interfejs użytkownika jest dostępny po uruchomieniu na: http://localhost:3000

T – Tempo – Obsługa danych związanych ze śledzeniem zdarzeń w programie (tracing) bazujący na grupowaniu wydarzeń w spany.

M – Mimir – Przechowuje metryki zbierane i przesyłane przy użyciu serwisu Prometheus. Interfejs Prometheusa jest dostępny po uruchomieniu na: http://localhost:9090

OpenTelemetry Collector – Jest centralnym punktem całego stacku. To do niego są przesyłane dane, które ten dystrybuuje do pozostałych elementów stacku LGTM oraz OpenTelemetry. Zajmuje dwa porty: `4317 – gRPC` oraz `4318 – HTTP`.

3. Przygotowanie konfiguracji Prometheusa.

Nasza aplikacja będzie ręcznie udostępniała dane metryk na endpoincie `/metrics`. Prometheus działa na zasadzie "pollingu" danych z tego endpointu, czyli wysyłaniu zapytań o nowe dane co określoną ilość sekund. Aby Prometheus wiedział, gdzie jest uruchomiona nasza aplikacja musimy skonfigurować jej adres. W tym celu utwórz plik `prometheus.yaml` i zapisz w nim następującą konfigurację:

```
global:
    scrape_interval: 5s
scrape_configs:
    - job_name: 'fastapi'
    static_configs:
            - targets: ['host.docker.internal:8000']
```

Uwaga: Ta konfiguracja zakłada, że nasza aplikacja FastAPI będzie uruchomiona na adresie: http://localhost:8000. Ustawienie hosta na `host.docker.internal` wskazuje na `localhost` na naszym komputerze. Jeśli korzystasz z systemu innego niż Windows, host wskazujący na `localhost` może wyglądać inaczej lub może być wymagane dodanie `--network host` do komendy uruchamiającej kontener. Zobacz więcej na temat wskazywania localhost z kontenera. Ustawiliśmy także port na którym jest uruchomiona aplikacja na `8000`. Jeśli korzystasz z innego, zmień tą wartość.
Wartość `scrape_interval` ustawiona na 5 sekund określa interwał czasowy w jakim Prometheus będzie sprawdzał endpoint `/metrics` w celu odczytu nowych metryk.

4. Uruchomienie stacku LGTM na Dockerze.

Upewnij się, że proces Docker Engine jest uruchomiony.
Uruchom terminal i przejdź do lokalizacji utworzonego pliku konfiguracyjnego `prometheus.yaml` używając polecenia `cd <ścieżka>`.

Wpisz komendę:

```
docker run --name lgtm -p 3000:3000 -p 3100:3100 -p 4317:4317 -p 4318:4318 -p 9090:9090 -v ./prometheus.yaml:/otel-lgtm/prometheus.yaml -ti grafana/otel-lgtm
```

Komenda pobiera kontener `grafana/otel-lgtm` ze stackiem LGTM, ustawia konfigurację

Prometheusa i eksponuje porty serwisów wchodzących w skład LGTM.

Po uruchomieniu komendy, na ekranie terminala powinno pojawić się podsumowanie czasu startu poszczególnych serwisów LGTM. Oznacza to, że stack LGTM działa poprawnie. Możesz upewnić się wchodząc do interfejsu Grafany: http://localhost:3000 i logując się domyślnym loginem i hasłem: admin / admin.

5. Pobieranie wymaganych zależności Python.

Zapiszemy listę wszystkich zależności do jednego pliku, aby łatwiej było pobrać wszystkie zależności na raz. Utwórz plik requirements.txt i zapisz do niego następujące nazwy zależności – każdy w osobnej linii.

```
prometheus_client
fastapi
uvicorn
python-logging-loki
opentelemetry-api
opentelemetry-sdk
opentelemetry-exporter-otlp
opentelemetry-instrumentation
opentelemetry-instrumentation-fastapi
```

Uruchom nowe okno terminala i przejdź do ścieżki utworzonego pliku requirements.txt. Pobierz wszystkie zależności komendą: pip install -r requirements.txt

6. Proste API z FastAPI korzystające ze stacku LGTM.

Utwórz plik main.py.
Zaimportuj zainstalowane, wymagane zależności:

```
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.resources import Resource
from prometheus_client import Summary, generate_latest, CONTENT_TYPE_LATEST
from fastapi.middleware.cors import CORSMiddleware
from fastapi import FastAPI, Response
from opentelemetry import trace
import logging_loki
import logging
import uvicorn
import random
import time
```

Tworzymy obiekt API i ustawiamy politykę CORS udostępniając je wszystkim hostom.

```
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
```

```
    allow_headers=["*"],
)
```

Konfigurujemy logi i łączymy je z Loki pracującym na Dockerze na porcie 3100.

```
LOKI_ENDPOINT = "http://localhost:3100/loki/api/v1/push"
logging.basicConfig(level=logging.INFO, format='[%(name)s] %(asctime)s - %(levelname)s - %(message)s -
trace_id=%(trace_id)s')

class TraceContextFilter(logging.Filter):
    def filter(self, record):
        span = trace.get_current_span()
        if span and span.get_span_context().is_valid:
            ctx = span.get_span_context()
            record.trace_id = format(ctx.trace_id, '032x')
        return True

def get_logger(name: str) -> logging.Logger:
    loki_logs_handler = logging_loki.LokiHandler(
        url=LOKI_ENDPOINT,
        tags={"app": name},
        version="1",
    )
    formatter = logging.Formatter('[%(name)s] %(asctime)s - %(levelname)s - %(message)s -
trace_id=%(trace_id)s')
    loki_logs_handler.setFormatter(formatter)

    logger = logging.getLogger(name)
    logger.setLevel(logging.INFO)
    logger.addHandler(loki_logs_handler)
    logger.addFilter(TraceContextFilter())
    return logger

api_logger = get_logger("lgtm-app")
```

Dzięki dodaniu handlera Loki, wszystkie logi utworzone za pomocą tego loggera zostaną przesłane do serwisu Loki i będą widoczne w interfejsie Grafany. Dodając TraceContextFilter będziemy mogli filtrować logi dla danego trace'a.

Konfigurujemy trace'y i spany, wskazujemy adres OpenTelemetry Collectora na który będą przesyłane wszystkie nasze trace'y.

```
trace_resource = Resource.create({
    "service.name": "lgtm-app"
})
trace.set_tracer_provider(TracerProvider(resource=trace_resource))
tracer_provider = trace.get_tracer_provider()
otlp_exporter = OTLPSpanExporter(endpoint="http://localhost:4318/v1/traces")
span_processor = BatchSpanProcessor(otlp_exporter)
tracer_provider.add_span_processor(span_processor)
tracer = tracer_provider.get_tracer(__name__)
```

Tworzymy własną metrykę mierzącą czas odpowiedzi API.

```
REQUEST_TIME_METRICS = Summary("request_processing_time", "Time spent processing the request.",
["endpoint"])
```

Obiekt Summary to jeden z możliwych typów metryk (zobacz więcej typów metryk). Pierwszy przekazany parametr to identyfikator metryki, drugi to opis, a trzeci to lista atrybutów metryki. W tym przypadku będziemy mogli filtrować wartości metryki po endpoincie na którym została ona zmierzona.

Ustawiamy endpointy naszego API

```
@app.get("/")
async def home():
    with REQUEST_TIME_METRICS.labels(endpoint="/").time():
        with tracer.start_as_current_span("span1"):
            sim_wait_time = random.randint(1, 5)
            api_logger.info(f"Simulating wait time: {sim_wait_time}s")
            time.sleep(sim_wait_time)
            api_logger.info(f"Finished waiting...")
            return f"Hello after {sim_wait_time}s"

@app.get("/metrics")
async def metrics():
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
```

Wejście na endpoint / spowoduje:

1. Zmierzenie czasu odpowiedzi serwera i zapisania go do metryk Mimir
2. Zapisanie zdarzeń w utworzonym spanie o nazwie span1 i zapisanie tych danych do Tempo.
3. Logowanie informacji o symulowanym czasie oczekiwania na ekran konsoli i zapisanie ich do Loki.
   Wszystkie zebrane dane będą dostępne w interfejsie Grafany.

Oprócz jawnie zbieranych przez nas danych, OpenTelemetry tworzy około 50 dodatkowych metryk.

Tak, jak zostało to opisane w punkcie 3, endpoint /metrics udostępnia dane o metrykach dla pollera Prometheusa.

Uruchomienie serwera HTTP z naszym API oraz jego instrumentacja.

```
FastAPIInstrumentor.instrument_app(app)
uvicorn.run(app, host="0.0.0.0")
```

7. Uruchomienie aplikacji.

Uruchom program komendą py main.py lub python3 main.py

8. Testowanie działania.

Odwiedź kilkukrotnie http://localhost:8000/ w celu wygenerowania danych. Zobacz je w interfejsie Grafany na: http://localhost:3000/ w kategorii Drilldown, zakładkach Logs, Metrics i Traces. Dane o metrykach z Prometheusa możesz zobaczyć oddzielnie na: http://localhost:9090/. Sprawdzisz tam też, czy Prometheus jest poprawnie skonfigurowany.

9. Automatyzacja śledzenia endpointów.

Żeby zautomatyzować śledzenie endpointów i trochę wyczyścić kod z powtarzających się wcięć związanych z mierzeniem czasu i trace'ów każdego endpointa, możemy dodać tzw.

middleware, czyli funkcji która jest wywoływana po odwiedzeniu endpointa, ale przed wykonaniem napisanej przez nas logiki związanej z tym endpoint'em. Działa to w taki sposób:

Odwiedzenie /metrics → Serwer HTTP otrzymuje zapytanie → Podstawowe przetwarzanie danych zapytania → **MIDDLEWARE** → Wywołanie naszego handlera tego endpointa: @app.get("/metric")...

Zdefiniujmy i dodajmy naszego middleware'a do śledzenia metryk we wszystkich endpoint'ach protokołu HTTP:

```
@app.middleware("http")
async def measure_response_time(request: Request, call_next):
    tracking_id = request.method + " /" + request.url.path.split("/")[1]
    with REQUEST_TIME_METRICS.labels(endpoint=tracking_id).time():
        with tracer.start_as_current_span(tracking_id + " - endpointSpan"):
            return await call_next(request)
```

Zmienna tracking_id służy do rozróżnienia zbieranych danych na endpointy i metody jakimi zostały one wywołane (np. GET /metrics)
Nasz middleware mierzy czas przetworzenia zapytania i zwrócenia odpowiedzi oraz grupuje wszystkie utworzone w nim span'y w span z nazwą endpointa.

10. Przykłady zastosowań obserwacji przy użyciu LGTM.

Logi, trace'y, a w szczególności metryki pozwalają nam zobaczyć nasz produkt z perspektywy pojedynczego użytkownika lub przeciętnego użytkownika (jeśli uśredniamy dane). Dane powinniśmy zacząć zbierać od samego początku publikacji produktu. Pozwoli nam to badać trendy i zmiany (np. zmianę zainteresowania daną częścią produktu po aktualizacji). Ilość i dokładność danych ustalamy sami definiując metryki, trace'y i logi tam, gdzie tego potrzebujemy. Warto jest dokładnie przemyśleć czego będziemy chcieli się dowiedzieć o naszym produkcie i w jaki sposób pozyskamy te dane jeszcze przed utworzeniem projektu.

1. **ZAŁOŻENIE:**
   Załóżmy, że nasza aplikacja umożliwia założenie konta. Korzystając z trace'ów i logów utworzyliśmy logikę endpointa odpowiedzialnego za tworzenie konta w następujący sposób:

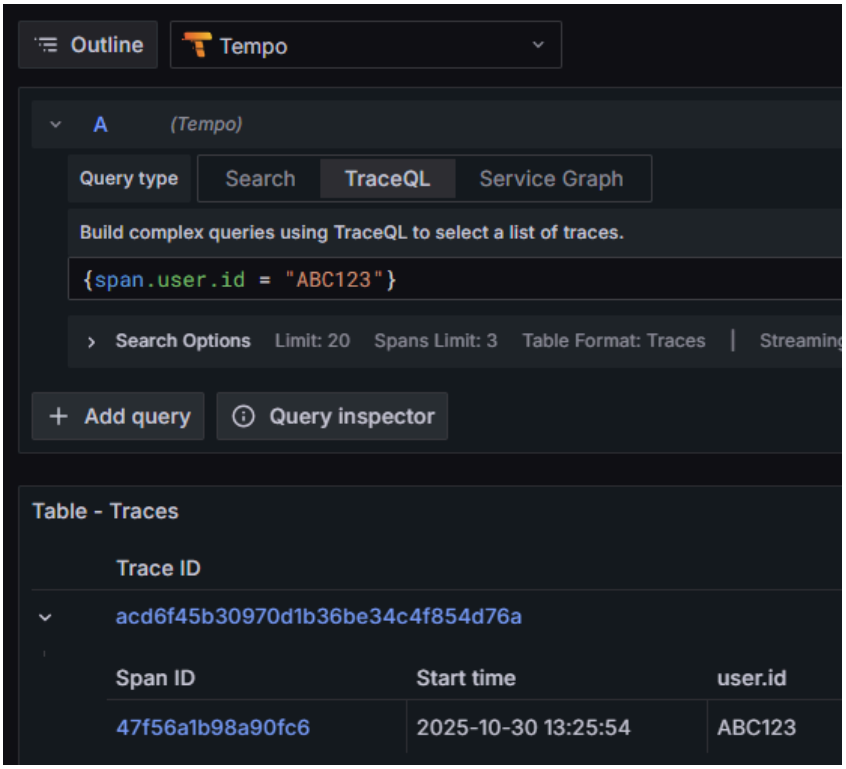
   ```
   with tracer.start_as_current_span("create-account", attributes={"user.id": uid}):
           api_logger.info(f"Validating data: `{uid}`")
           validate_data(uid)
           api_logger.info(f"Saving account to DB: `{uid}`")
           save_to_db(uid)
           api_logger.info(f"Successfully created account: `{uid}`")
   ```

   Zauważ, że dodaliśmy atrybut user.id do spanu create-account, a przed i po każdym etapie tworzenia konta tworzymy logi.

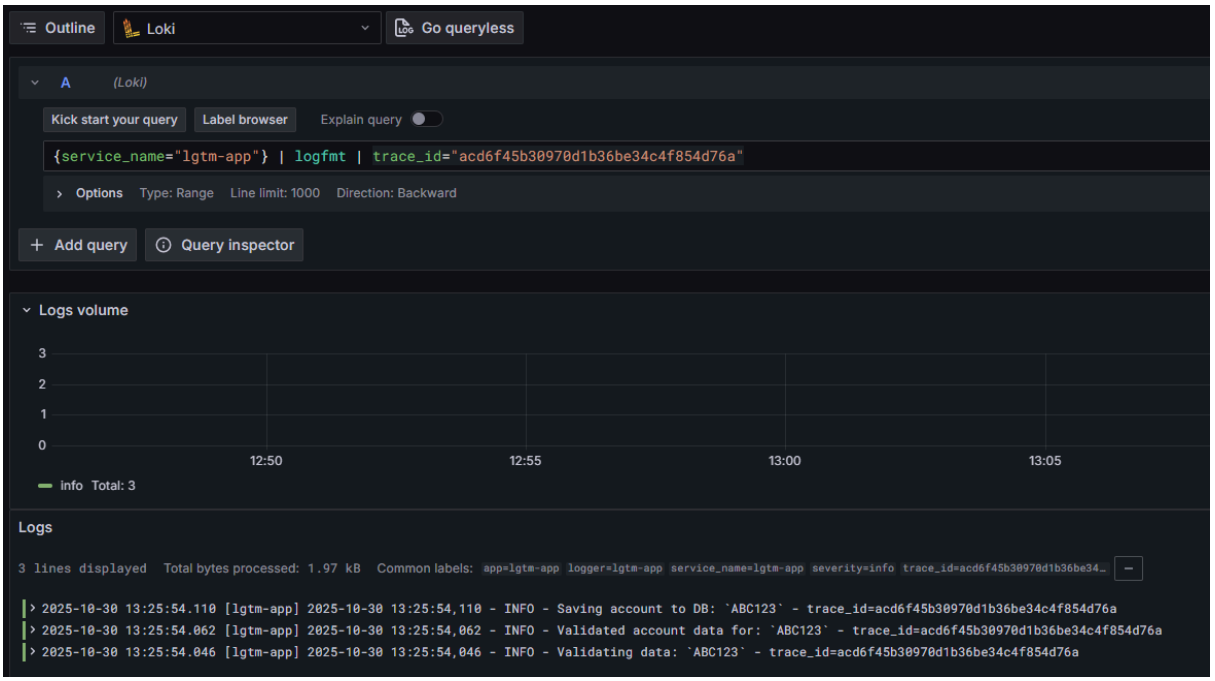
ZASTOSOWANIE:

Użytkownik zgłasza nam problem z utworzeniem konta. Na ekranie widzi błąd informujący go o niepowodzeniu utworzenia konta o id `ABC123`

Przejdź do zakładki Explore interfejsu Grafany i wyszukaj span'ów z atrybutem `user.id="ABC123"`:



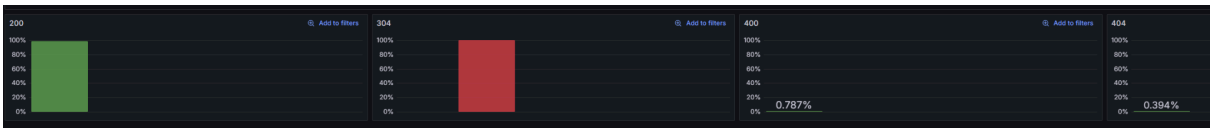
Widoczny tutaj Trace ID to trace utworzony podczas nieudanego tworzenia konta. Skopiujmy ID i wyszukajmy logów, które powstały podczas tego trace'a.



Zmień źródło danych na Loki i przepisz skrypt TraceQL do wyszukania logów. Zauważ, że nie pojawia się tutaj ostatni log, którego oczekujemy (`Successfully created account: `{uid}``), ale widzimy poprzedni: (`Saving account to DB: `{uid}``). Oznacza to, że błąd wystąpił pomiędzy tymi logami – w funkcji `save_to_db(uid)`.

2. Implementując najprostszą metrykę mierzącą czas odpowiedzi danego endpointu (jak w przykładzie powyżej) możemy nie tylko obserwować zmiany w czasie odpowiedzi serwera na dane zapytanie i reagować implementując np. load balancer, ale też zobaczyć w jakich godzinach najczęściej używany jest dany endpoint – co przekłada się na aktywność użytkowników.

3. Korzystając konfiguracji środowiska śledzącego aplikację, które napisaliśmy powyżej, zbierane są też dane np. o zwracanych kodach statusu HTTP. Wcodząc w zakładke traces → comparison i wybierając http.status_code możemy zobaczyć porównanie ilości zwracanych kodów. Znaczny wzrost błędnych odpowiedzi oznacza, że powinniśmy zareagować. Być może mamy gdzieś błąd w kodzie, który możemy znaleźć korzystając z trace'ów zapytań z odpowiedziami z kodami błędu lub logach. Może się też okazać, że użytkownicy nie rozumieją, w jaki sposób powinni używać danej funkcji naszego produktu, bo jest stworzona w nieintuicyjny sposób i powinniśmy to naprawić. W przypadku większych projektów nawet względnie niski procent niezadowolonych klientów może równać się z dużymi stratami finansowymi, dlatego należy doszukiwać się wszelkich niedogodności.



4. Porównując kraje, z których pochodzą nasi klienci możemy lepiej zaplanować kampanię reklamową tak, aby celować w miejsca, gdzie zauważamy większy popyt na nasz produkt. Bez informacji, skąd pochodzą nasi klienci ciężko byłoby zaplanować optymalną kampanię.